FRONTEND INTEGRATION · SOCKET.IO

TeleBook Socket Contract

Every socket event the backend sends and accepts, what each one expects, and the exact client code for connecting, messaging, read ticks and push notifications. Following the five rules below is what makes online status and read ticks behave correctly.

SERVER	CLIENT	AUTH	BACKEND VERSION
https://api.telebookservices.com	socket.io-client v4 · websocket	JWT in handshake	18 Sep 2026

THE FIVE RULES

1. **Connect with the JWT in the handshake** — `io(URL, { auth: { token } })`.
   The server identifies the user from the token, puts the socket in that user's room and marks them online. No `setup` event is needed.
2. **Send messages over REST only** — `POST /api/messages`. Never emit `new-message`.
   The server delivers to every recipient and sends push notifications itself. The socket `new-message` event is now ignored.
3. **On every `message-received`, emit `message-delivered`**; if that chat is open on screen, also emit `message-read`.
   This is what turns the sender's single tick into double grey and then double blue.
4. **Disconnect the socket when the app goes to background**, reconnect when it becomes active.
   Push notifications are sent only when a user has no live socket. A socket left open in background means the user looks online and receives no notifications.
5. **Emit `join-chat` when a chat screen opens and `leave-chat` when it closes** — and re-join after every reconnect.
   Typing indicators, deletions, edits and reactions are sent only to sockets inside the chat room.

WHAT CHANGED ON THE BACKEND

Two reported issues were caused on the server and are fixed there. Nothing in the app needs to change for the fixes to take effect, but the rules above are what let them work fully.

ISSUE	CAUSE	NOW
Push notifications arrived while the user was online	Online status expired after 30 s unless the app sent `heartbeat` every 25 s. A reconnect or a second device could also wipe it.	The server tracks every open socket itself. A user is online while any socket is connected. `heartbeat` is no longer needed.
Sender stayed on a single tick after the receiver read the message from another screen	Opening a chat loads messages over REST, which saved the read but never told the sender.	`GET /api/messages/:chatId` and `PUT /api/messages/:id/read` now emit `message-read-update` to the sender, the same event the socket path sends.

CONNECTING

One socket for the whole app, created after login. This service handles authentication, reconnects, the background rule and re-joining the open chat.

```
// services/socket.js
import { io } from 'socket.io-client';
import { AppState } from 'react-native';

const SOCKET_URL = 'https://api.telebookservices.com';

let socket = null;
let appStateSub = null;
let openChatId = null;           // the chat currently on screen

export function connectSocket(token) {
  if (socket) return socket;

  socket = io(SOCKET_URL, {
    transports: ['websocket'],
    auth: { token },                  // rule 1 — server reads the user from this
    reconnection: true,
    reconnectionDelay: 1000,
    reconnectionDelayMax: 10000,
  });

  socket.on('connect', () => {
    // Rooms are per socket. After any reconnect the chat room is gone —
    // join it again (rule 5). The personal room is rejoined by the server.
    if (openChatId) socket.emit('join-chat', openChatId);
  });

  socket.on('connected', () => { /* server has marked this user online */ });
  socket.on('connect_error', (err) => console.warn('socket:', err.message));

  // Rule 4 — background means offline, so push notifications reach the user
  appStateSub = AppState.addEventListener('change', (state) => {
    if (state === 'active') {
      if (!socket.connected) socket.connect();
    } else if (state === 'background') {
      socket.disconnect();            // not on 'inactive' — iOS enters that briefly
    }
  });

  return socket;
}

export function disconnectSocket() {     // call on logout
  appStateSub?.remove();
  socket?.disconnect();
  socket = null;
  openChatId = null;
}

export const getSocket = () => socket;
export const setOpenChat = (chatId) => { openChatId = chatId; };
export const getOpenChat = () => openChatId;
```

> **Why disconnect instead of staying connected in background:** the backend sends a push only to users with no live socket. Disconnecting is also what sets *last seen* correctly.

MESSAGE LIFECYCLE

The full path of one message from A to B, and which side does what.

1	**A** calls `POST /api/messages` with `{ chatId, content, messageType }`. The response is the saved message — show it with a single tick.
2	**Server** emits `message-received` to every other member. Members with no live socket also get a push notification.
3	**B** receives `message-received` and emits `message-delivered` — even if B is on another screen.
4	**Server** emits `message-delivered-update` to A. A shows double grey.
5a	**B**, if that chat is already open: emits `message-read`.
5b	**B**, if B opens the chat later: calls `GET /api/messages/:chatId`. The server marks every unread message as read — nothing to emit.
6	**Server** emits `message-read-update` to A for each message, in both 5a and 5b. A shows double blue.

Sending

```
const { data } = await api.post('/api/messages', {
  chatId, content, messageType: 'text',   // 'text' | 'image' | 'video' | 'audio' | 'document' | 'emoji'
  replyTo,                                // optional message _id
  attachments,                            // optional — upload first via POST /api/users/media
});
addToChat(data.message);                  // single tick
// Do NOT emit 'new-message'. The server has already delivered it.
```

Receiving

```
import { getSocket, getOpenChat } from './socket';
import { AppState } from 'react-native';

socket.on('message-received', (message) => {
  const chatId = message.chat?._id ?? message.chat;

  socket.emit('message-delivered', { messageId: message._id });          // always — rule 3

  const onScreen = chatId === getOpenChat() && AppState.currentState === 'active';
  if (onScreen) {
    socket.emit('message-read', { messageId: message._id, chatId });      // only when actually visible
  }

  upsertMessage(message);                 // chat screen, chat-list preview, unread badge
});
```

Opening and closing a chat

```
export async function openChat(chatId) {
  setOpenChat(chatId);
  getSocket()?.emit('join-chat', chatId);                    // rule 5
  const { data } = await api.get(`/api/messages/${chatId}`);
  // This call marks all unread messages as read and notifies their senders.
  // It also resets this chat's unseenCount to 0 on the server.
  return data;
}

export function closeChat(chatId) {
  getSocket()?.emit('leave-chat', chatId);
  setOpenChat(null);
}
```

Showing ticks

For messages you sent. Update the message's `deliveredTo` / `readBy` arrays when the update events arrive, then derive the tick from them:

✓	✓✓	✓✓
Sent	**Delivered**	**Read**
saved; no recipient in deliveredTo	every recipient in deliveredTo	every recipient in readBy

```
socket.on('message-delivered-update', ({ messageId, userId, chatId }) =>
  addReceipt(messageId, 'deliveredTo', userId));

socket.on('message-read-update', ({ messageId, userId, chatId }) => {
  addReceipt(messageId, 'readBy', userId);
  addReceipt(messageId, 'deliveredTo', userId);   // read implies delivered
});

export function tickState(message, myId, chatUsers) {
  const others = chatUsers.map((u) => String(u._id)).filter((id) => id !== String(myId));
  const has = (list, id) => (list || []).some((r) => String(r.user) === id);
  if (others.every((id) => has(message.readBy, id)))      return 'read';
  if (others.every((id) => has(message.deliveredTo, id))) return 'delivered';
  return 'sent';
}
```

In groups, `every` means blue ticks appear once all members have read. Use `some` if the product wants blue on first read.

EVENT REFERENCE

`APP → SERVER` you emit `SERVER → APP` you listen `DEPRECATED` remove

App → Server

EVENT	PAYLOAD	WHEN
message-delivered	`{ messageId }`	On every `message-received`
message-read	`{ messageId, chatId }`	Message received while its chat is on screen
join-chat	`chatId`	Chat screen opens; again after each reconnect
leave-chat	`chatId`	Chat screen closes
typing	`{ chatId }`	User starts typing
stop-typing	`{ chatId }`	Stops typing or sends
get-user-status	`{ userId }`	Opening a chat, to show online / last seen
delete-message	`{ messageId, chatId }`	Relay only — the REST delete already notifies
edit-message	`{ messageId, chatId, newContent }`	Edit your own message — saved (same as `PUT /api/messages/:id`)
message-reaction	`{ messageId, chatId, emoji }`	After `PUT /api/messages/:id/react` succeeds
user-added-to-group	`{ chat, newUserId }`	After `PUT /api/chats/group/add`
user-removed-from-group	`{ chat, removedUserId }`	After `PUT /api/chats/group/remove`
group-info-updated	`{ chat }`	After rename / description change
new-status	`statusData`	After `POST /api/status`
join-channel	`channelId`	Channel screen opens
leave-channel	`channelId`	Channel screen closes
call-user	`{ to, offer, from, callType }`	Start a call — `to` is a userId
answer-call	`{ to, answer }`	Accept
ice-candidate	`{ to, candidate }`	WebRTC ICE
end-call	`{ to }`	Hang up
reject-call	`{ to }`	Decline
new-message DEPRECATED	—	Ignored by the server. Remove it.
heartbeat DEPRECATED	—	No longer needed. Safe to remove.
setup LEGACY	`{ _id }`	Only without a token. With a token it is optional and the `_id` is ignored.

Server → App

The *Reaches* column matters: events sent to a chat room arrive only if you emitted `join-chat`.

EVENT	PAYLOAD	REACHES	MEANING
connected	–	this socket	User marked online
message-received	`message (populated sender, chat)`	user room	New message, also group-created system message
message-delivered-update	`{ messageId, userId, chatId }`	sender	A recipient got the message
message-read-update	`{ messageId, userId, chatId }`	sender	A recipient read the message
user-online	`userId`	everyone	User came online
user-offline	`userId`	everyone	User's last socket closed
user-status-result	`{ userId, status }`	this socket	Reply to `get-user-status`; `'online' | 'offline'`
typing	`{ chatId, userId }`	chat room	Someone is typing
stop-typing	`{ chatId, userId }`	chat room	Stopped typing
messages-deleted	`{ messageIds, chatId }`	chat room	From `DELETE /api/messages`
message-deleted	`{ messageId, chatId }`	chat room	From the `delete-message` relay
message-edited	`{ messageId, newContent, chatId, editedAt }`	chat room	A message was edited
edit-message-error	`{ messageId, message }`	this socket	Your edit was refused
message-reaction-update	`{ messageId, emoji, userId }`	chat room	Reaction added
added-to-group	`chat`	added user	You were added
removed-from-group	`chatId`	removed user	You were removed
group-updated	`chat`	members	Members or group info changed
status-update	`statusData`	everyone	New status posted
channel-message-received	`channel message`	channel room	New channel post
channel-message-deleted	`{ channelId, messageId }`	channel room	Post deleted
incoming-call	`{ from, offer, callType }`	callee	Ring
call-answered	`answer`	caller	Callee accepted

EVENT	PAYLOAD	REACHES	MEANING
ice-candidate	candidate	peer	WebRTC ICE
call-ended	—	peer	Other side hung up
call-rejected	—	caller	Callee declined

> **Edits are saved.** `edit-message` (or `PUT /api/messages/:id`) updates the message for everyone and marks it `isEdited`. Reactions are saved by `PUT /api/messages/:id/react` — call that first, then emit `message-reaction`.

PRESENCE AND TYPING

```
socket.on('user-online',  (userId) => setPresence(userId, 'online'));
socket.on('user-offline', (userId) => setPresence(userId, 'offline', new Date()));

// When a chat opens, ask once — don't wait for a broadcast
socket.emit('get-user-status', { userId: otherUserId });
socket.on('user-status-result', ({ userId, status }) => setPresence(userId, status));

// Typing — debounce so it isn't sent on every keystroke
let typingTimer;
function onChangeText(text) {
  if (!typingTimer) socket.emit('typing', { chatId });
  clearTimeout(typingTimer);
  typingTimer = setTimeout(() => { socket.emit('stop-typing', { chatId }); typingTimer = null; }, 2000);
}
socket.on('typing',      ({ chatId, userId }) => showTyping(chatId, userId, true));
socket.on('stop-typing', ({ chatId, userId }) => showTyping(chatId, userId, false));
```

A user counts as online while any of their devices has a live socket. Closing one of two phones does not mark them offline.

PUSH NOTIFICATIONS

The server sends a push only when the recipient has **no live socket** — the app is in background (rule 4), closed, or offline. While the app is open, messages arrive as `message-received` instead.

Register the device token

```
// At login
await api.post('/api/auth/verify-otp', {
  userId, otp,
  fcmToken, deviceType: Platform.OS,       // 'android' | 'ios' | 'web'
  deviceId,                                // stable per install — replaces this device's old token
});

// Whenever Firebase rotates the token
messaging().onTokenRefresh((token) =>
  api.post('/api/notifications/save-token', { token, deviceType: Platform.OS, deviceId }));
```

Payload

FIELD	EXAMPLE	USE
notification.title	"Rahul" / "Team • Rahul"	Sender, or group name and sender
notification.body	"hello" / "📷 Image"	Text preview, or media type
data.entity	"chat"	Screen type to open
data.entityId	"673a1f8c…"	Chat id — navigate to it on tap
data.senderId	"673a1f8c…"	
data.senderName	"Rahul"	
data.isGroup	"true" / "false"	A *string* — compare with `=== 'true'`

> **Force-closed app:** if the OS kills the app without closing the socket, the server needs up to about 85 seconds to notice. Messages in that window arrive without a push. Normal backgrounding does not have this gap because rule 4 closes the socket immediately.

COMMON MISTAKES

> **Emitting `new-message` after sending.** The server ignores it — by the time the REST call returns, every recipient has already been sent the message. Leaving it in only adds traffic.

> **Not re-joining the chat room after a reconnect.** Typing, deletes and reactions stop arriving silently. The `connect` handler in the service above takes care of it.

> **Emitting `message-read` for a chat that isn't visible.** The sender sees blue ticks for a message nobody read. Check the open chat and `AppState.currentState === 'active'`.

Keeping the socket connected in background. The user looks online and gets no push notifications.

Reading `data.isGroup` as a boolean. FCM data values are strings; `"false"` is truthy.

TEST BEFORE RELEASE

Two phones, users A and B. Each item should pass.

- [ ] A and B both open, B on the chat list. A sends a message.
  B gets no push notification. B's chat list updates. A sees double grey.
- [ ] B then opens the chat.
  A's ticks turn blue within about a second.
- [ ] Both on the same chat screen, chatting.
  Blue ticks immediately; typing indicator shows on both sides.
- [ ] B sends the app to background. A sends a message.
  B gets a push. Tapping it opens that chat.
- [ ] B turns on airplane mode for 20 seconds, then off.
  B reconnects on its own; typing and reactions still work in the open chat.
- [ ] B signed in on two phones; close the app on one.
  A still sees B as online.